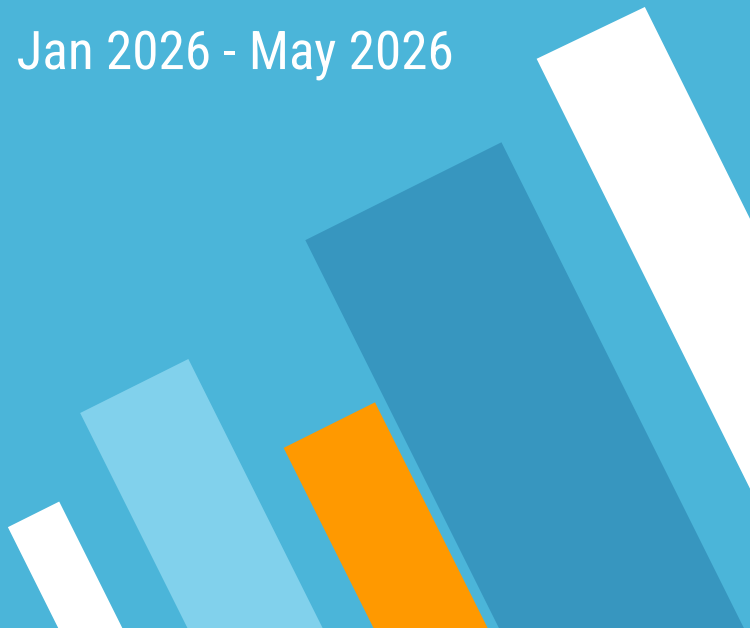




SmartBank: Design and Implementation of a Secure SaaS-Style Digital Banking Platform

Vignesh Sriram
Binghamton University
MS in Information Systems
B-Number: B00811333
Advisor: Professor Xin Zhang

Jan 2026 - May 2026



What is SmartBank?

SaaS-style digital banking prototype

Built using Flask and SQLite

Simulates core banking operations:

- User authentication
- Deposits / withdrawals
- Account upgrades (Free vs Pro)
- GitHub OAuth login

Demonstrates fintech + information systems principles

The goal of this project is to bridge cybersecurity, SaaS, and fintech concepts into one prototype.



Motivation



Why I Built This

- Inspired by earlier cybersecurity banking project at St. John's University during my Secure Software Development class
- Expanded it into a SaaS-oriented fintech system
- Interest in:
 - Cloud computing
 - Secure system design
 - Fintech applications
- Opportunity to apply theory to a practical system, my plans is to be a solution engineer in the future. With actively studying the AWS CLF C02 and being in the Tech Fellow at Codepath, I hope to apply these skills in the future.

**CODE
PATH
*ORG**



Motivation Pt 2

Why I Built This

- Cybersecurity → authentication, hashing, secure storage
 - Information Systems → database + workflows + users
 - Solution Engineering → designing a system to solve a business problem
- 



Problem Statement

Challenge

Modern fintech systems require:

- Secure authentication
- Scalable architecture
- Reliable transaction processing
- User-centric service models

Problem

Many learning projects show pieces of this, but not a unified SaaS-style banking system.

Solution

GeniusBank provides a prototype integrating these concepts.

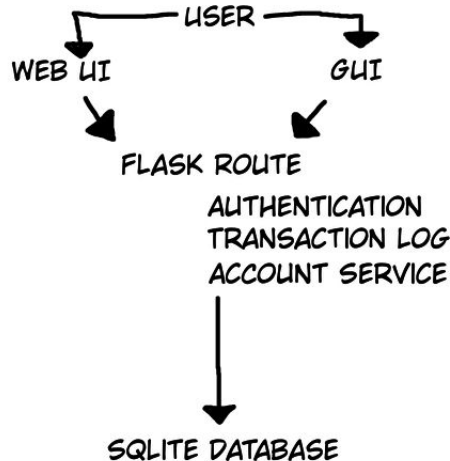


System Architecture

Architecture Components

- Client Layer (Web + GUI)
- Flask Backend
- Authentication Layer
- Transaction Services
- SQLite Database

Architecture: Client-Server / Service-Oriented



Database Design (ER Diagram)

Entities:

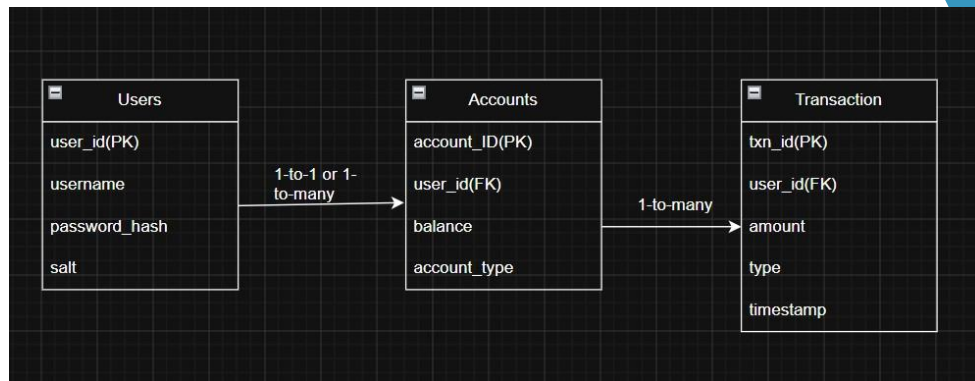
- Users
- Accounts
- Transactions

Purpose:

- Data isolation
- Transaction consistency
- Relational structure

Relationships:

- User $\rightarrow$ Account (1:1)
- User $\rightarrow$ Transactions (1:M)



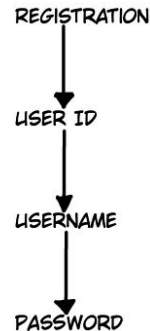
Core Features

Implemented Functionality

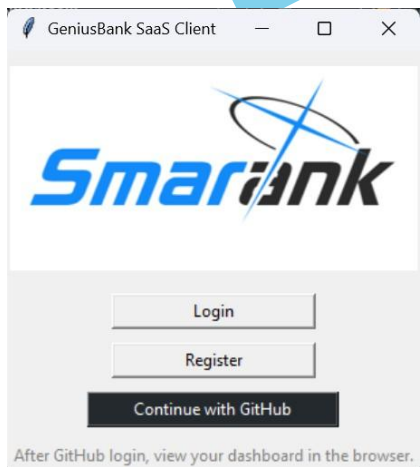
- Signup / Login
- GitHub OAuth authentication
- Deposits
- Withdrawals
- Free vs Pro account tiers
- Upgrade / downgrade options
- Session-based user access control

Other Functions

- Custom Exceptions such as daily exceptions
- Security Handling



GUI and Web Interface



GeniusBank

Login

Or login with

Don't have an account?

- » Login/Register page
- » GitHub OAuth flow
- » GUI dashboard
- » Logged-in banking interface

Key point: Same backend logic supports multiple interfaces.

- Red - user_id
- Blue - username
- Green - password hash
- Yellow - salt

Password -> 123 -> output says random stuff, that is hash.
Salt is something unique with if same password.

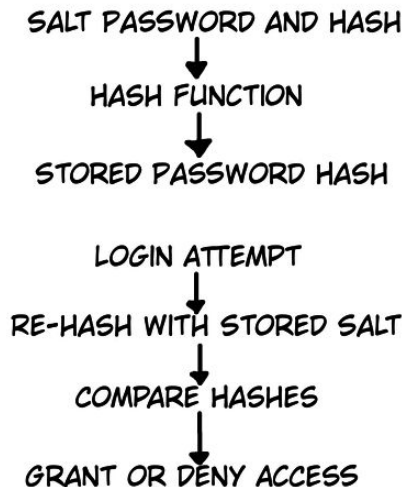
If User A: $123 + X7$
User B: $123 + A9$

Even if both have 123 password, they have different final box

- Password hashing + salting
- Session-based authentication
- Backend validation
- Protected routes

Why do you need protections and what are some examples?

- Prevent overdrafts
- Prevent unauthorized actions
- Reduce credential exposure



```
'\vigñesh', 'vigñesh', b'\x65\x0c\x69\xca\x0a \x04\xef\xbf\r\x04\xf5\x1b\x0a\x3\r\x0b5\xa2\xadq5bf\x02p\xfb\x0a7\xcd\x0a0c', b'\xb7\x0a5\x0a\x09
82m\x95\x0c\x0e5H\x0ab\x9d96c')
('', '', b'\xb2\x01\x9e\x04d\x0b\x0c\xf5f?x\dc\x0a85H\x0b5\x09c\xef\x0b1\x080\x0d0\x04\xbb\x0ecH\x0b\x07f\x0b9D_\xdf', b's?x\fa\x0f1\x09c2\x0b7\x0f5\x0f1
x80FTV, &')
('dog', 'dog', b'\xb59\x0c\x04\x09\x0d0\x0a\x06eY\x0c\x05v8\x09\r\x0b3\x0fe\x05\x0deyP\r\x0e3\r\x0182\x0d\x085c', b'\xa3\x0b3q\x0ca\x02\reff\x086n\x0e0:\x0e:\x0e:
\xff\x0f3W\x0ca')
('Ichigo22', 'Ichigo22', None, None)
('Vig270', 'Vig270', None, None)
('solar', 'solar', b'\xcd\x0dfm\x0c085\x0b\x0c8\x0a2\x0aa\x058j0\r\x0be<\x01CQ\x09\x06\x0cea\x0ef\x089\x041t\xfd\x0b\x0d3', b'y'\x0a\x0c3\x0ca\x058;~1\x
d9H\x08a\x07~093P\x0f7U\x0e3\x13R\x168S\x0c\x0e1\x14a\x0f\x0c5\x08f')
('smart', 'smart', b'@\x0a\x0b\x0c\x0a\x0c\x016\x05\x0b7\x0ba\x04\x0d\x07"#\x0adT\x09N\x0c\x0d6\x080\x0e1\x093pL\x0d7\x091', b'p\x086\x0d2\x0f4f\x092d\x108\x0e1\x18$
071\x080\x0eap\x10a\x0e1\x14\x0ebfu\x0902\x0d7\x0f4\x086\x082\xff\x096\x0e1')
('gamma', 'gammal23', b'\x931\x0c\x085\x08b\x0f\x0131"'\x086\x086\x0eb\x0d3\x0b8\x0f5Z\x089\x0b\x0ba0Q\x0a3\x0a3\x0c3\x0ede\n\x0ed\x07f\x087\x094', b'\x0e3\x0bbv\x
083\x086\x0cdw,\x0t\x0d2 \x0ad\r\x086\x08b\x0d4-j\x0ed9\x0d3c3~0\x09e\x0c0e')
('alpha', 'alphaTest', b'\xa8\x0e90\x0f8m\x14a\x08\x0e\x03c~.Z-\x089\x14d\x0b2\x0c6\x0b5\x086\x082\x09d\x0d2\x08a\x0aa\x0ddhb\x0d2c '\x09'\x0f', b'\x09d\x0d. \x0a
5\x096\x0e\x0d9\x08a\x0b\x0ea\x04d\r\x0a731'\x090\x0a8', S'f2>\x07\x0f7d\x0b6q\x0b3\x0ec?X\x0c5')
('beta', 'betaUser', b'\x0e6E2?x1880\x0fem\x012\x0f4t\x0f4'\x0a3\n6\x095\x1c0qM\x0dc/\x039~\x0a8\x0a7\x0a7\x0a8')
('f4\x081\x0be\x0f\x0e6\x0f6_8Rx\x0b6\x0aot\x108\x0bd2')
('delta', 'deltaDev', b'\xbdb\x01c\x089PTRK\r\x09X\x0bf\x0bbj; \x0q\x08f\x0b3\x0b\x09f\x09\x0d\x0f\x0c\x0eI\x0d\x0ad', b'\x0d0\x1c1\x0b1\4\x015'\x0b\x0c\x0f\x0d4\x0
cd\x0a3\x18m6\x0daf\x0da\x0a\x0d6\x0e3\x0b2\x0c\x0f\x0a5\x0bf\x0b\x080')
('epsilon', 'epsilonX', b'\x080\x0ca\x0f9\x0afu\x0f01\x0bd\r\x04\x0e\x097x\x070\r\x0f\x05\x0c6\x01\x0e\x0b\x05bH\x0c5\x0c0\x0a8\x03#x0d9p5', b'7\x089
cE\x084\x0d0\x0e\x0c\x08\x092\x05b7\x0a0\x087\x0b38\ns'H\x02\x14\x0e\x098\x0d2\x0c1\x089\x080')
('thea', 'thet99', b'\x084PK\x0ec\x0b0\x088,\x0d3M\x0cd\x019\x0a4\x0bb\x010\x0d5\x08c\x0945\x0ca\x09f\x096\x0e1\x0f\x0eH\x099'\x08c\x0d.', b'\x12t1@\x014\x0e48q
xc25\xf8\x090\x0e6\x0f6\x0e3:\x0e\x0b2j\x0fc2\x0ae\x096P=\x088'\x0b7\x08f\x0b5b')
```

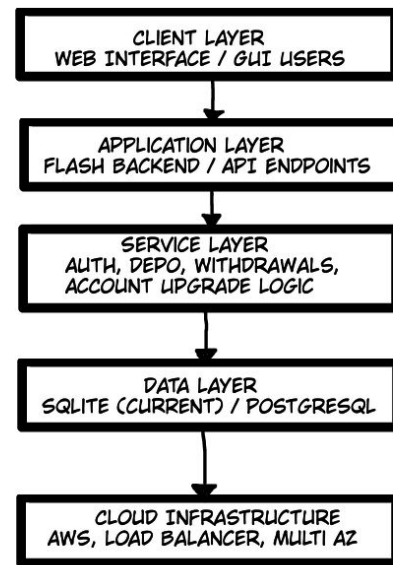
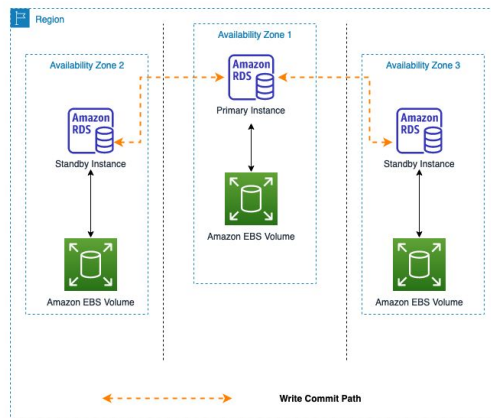
SaaS and Real World Experience

SaaS Concepts Demonstrated

- Multi-user system
- Tiered service model
- Centralized application logic
- Cloud-ready architecture principles

What are some Potential AWS deployment?

- Load balancing
- Multi-AZ
- PostgreSQL
- Scalable cloud hosting



Related Works

Compared To

- Traditional digital banking platforms
- SaaS fintech platforms (PayPal / Stripe)
- Educational Flask banking projects

Difference with GeniusBank

- Combines:
 - Authentication
 - SaaS model
 - Banking simulation
 - Security controls
 - Cloud-oriented design

Findings

Key Findings

- Lightweight Flask architecture successfully supports prototype banking workflows
- Modular design improved maintainability
- Hashing + salting strengthened credential security
- SQLite works for prototyping, but limits scalability

Overall, the **major takeaway** of this is the prototype demonstrates practical fintech system design principles.

Challenge and Limitations

Challenges:

Some of the challenges lead to these several things such as

- Session management issues during development
- Feature integration without breaking existing logic
- SQLite scalability limits
- OAuth provider constraints

Limitations:

- No fraud detection
- No external banking APIs
- Prototype-scale database
 -

Future Improvements

Planned Enhancements

- AWS deployment
- PostgreSQL migration
- MFA
- Transaction auditing
- External financial APIs
- Fraud detection / ML analytics
- Mobile-ready interface
- Move the Github Auth to Google Auth by paying the google services
- Appealing UI interface in both web and gui mode - can be even more unique

Goal: Move from prototype → production-ready platform

○

Main Contributions

This Project Demonstrates

- Fintech system modeling
- Secure authentication design
- SaaS architecture concepts
- Database-backed transaction processing
- Cloud-ready architectural thinking



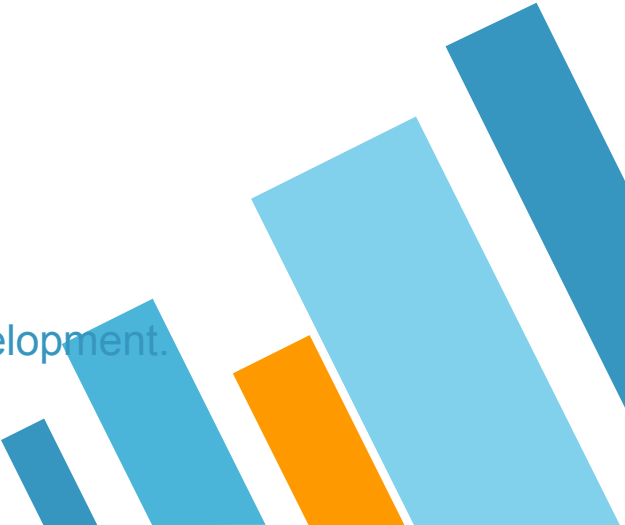
Conclusion

GeniusBank shows how:

- Information systems concepts
- Secure software design
- SaaS principles
- Fintech functionality

can be integrated into a cohesive banking prototype.

Result: A scalable foundation for future cloud-based fintech development.





Demo

Totally not scripted but go with flow if you know how





QA

Any Questions

